



Cyberscope

A *TAC Security* Company

Audit Report

Nummus

June 2026

Network POL

Address 0xCF275D9b4C7b4681ea8C5A14aFfCd687136a6490

Audited by © cyberscope

Table of Contents

Table of Contents	1
Analysis	4
Diagnostics	5
Risk Classification	8
Review	9
Audit Updates	9
Source Files	9
Findings Breakdown	10
ST - Stops Transactions	11
Description	11
Recommendation	12
MDD - Missing Dividend Distribution	13
Description	13
Recommendation	13
SLB - Staking Lock Bypass	14
Description	14
Recommendation	15
TSD - Total Supply Diversion	16
Description	16
Recommendation	16
ZD - Zero Division	17
Description	17
Recommendation	17
ISA - Inaccurate Swap Amount	18
Description	18
Recommendation	18
SFB - Staking Fee Bypass	19
Description	19
Recommendation	19
AOI - Arithmetic Operations Inconsistency	20
Description	20
Recommendation	20
CCR - Contract Centralization Risk	21
Description	21
Recommendation	22
DDP - Decimal Division Precision	23
Description	23
Recommendation	24
LED - Lacking Event Details	25

Description	25
Recommendation	25
INC - Inconsistent Naming Convention	26
Description	26
Recommendation	26
MC - Missing Check	27
Description	27
Recommendation	27
MEM - Missing Error Messages	28
Description	28
Recommendation	28
MAL - Misused Algorithmic Logic	29
Description	29
Recommendation	29
PVC - Price Volatility Concern	30
Description	30
Recommendation	30
RCS - Redundant Code Segment	31
Description	31
Recommendation	31
RED - Redundant Event Declaration	32
Description	32
Recommendation	32
RSML - Redundant SafeMath Library	33
Description	33
Recommendation	33
RSK - Redundant Storage Keyword	34
Description	34
Recommendation	34
RS - Redundant Swaps	35
Description	35
Recommendation	35
RVD - Redundant Variable Declaration	36
Description	36
Recommendation	36
SBE - Staking Bonus Expiry	37
Description	37
Recommendation	37
TUU - Time Units Usage	38
Description	38
Recommendation	38
L02 - State Variables could be Declared Constant	39

Description	39
Recommendation	39
L04 - Conformance to Solidity Naming Conventions	40
Description	40
Recommendation	41
L05 - Unused State Variable	42
Description	42
Recommendation	42
L07 - Missing Events Arithmetic	43
Description	43
Recommendation	43
L09 - Dead Code Elimination	44
Description	44
Recommendation	45
L12 - Using Variables before Declaration	46
Description	46
Recommendation	46
L13 - Divide before Multiply Operation	47
Description	47
Recommendation	47
L14 - Uninitialized Variables in Local Scope	48
Description	48
Recommendation	48
L15 - Local Scope Variable Shadowing	49
Description	49
Recommendation	49
L16 - Validate Variable Setters	50
Description	50
Recommendation	50
L19 - Stable Compiler Version	51
Description	51
Recommendation	51
L20 - Succeeded Transfer Check	52
Description	52
Recommendation	52
Functions Analysis	53
Inheritance Graph	59
Flow Graph	60
Summary	61
Disclaimer	62
About Cyberscope	63

Analysis

● Critical ● Medium ● Minor / Informative ● Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Unresolved
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	MDD	Missing Dividend Distribution	Unresolved
●	SLB	Staking Lock Bypass	Unresolved
●	TSD	Total Supply Diversion	Unresolved
●	ZD	Zero Division	Unresolved
●	ISA	Inaccurate Swap Amount	Unresolved
●	SFB	Staking Fee Bypass	Unresolved
●	AOI	Arithmetic Operations Inconsistency	Unresolved
●	CCR	Contract Centralization Risk	Unresolved
●	DDP	Decimal Division Precision	Unresolved
●	LED	Lacking Event Details	Unresolved
●	INC	Inconsistent Naming Convention	Unresolved
●	MC	Missing Check	Unresolved
●	MEM	Missing Error Messages	Unresolved
●	MAL	Misused Algorithmic Logic	Unresolved

●	PVC	Price Volatility Concern	Unresolved
●	RCS	Redundant Code Segment	Unresolved
●	RED	Redundant Event Declaration	Unresolved
●	RSML	Redundant SafeMath Library	Unresolved
●	RSK	Redundant Storage Keyword	Unresolved
●	RS	Redundant Swaps	Unresolved
●	RVD	Redundant Variable Declaration	Unresolved
●	SBE	Staking Bonus Expiry	Unresolved
●	TUU	Time Units Usage	Unresolved
●	L02	State Variables could be Declared Constant	Unresolved
●	L04	Conformance to Solidity Naming Conventions	Unresolved
●	L05	Unused State Variable	Unresolved
●	L07	Missing Events Arithmetic	Unresolved
●	L09	Dead Code Elimination	Unresolved
●	L12	Using Variables before Declaration	Unresolved
●	L13	Divide before Multiply Operation	Unresolved
●	L14	Uninitialized Variables in Local Scope	Unresolved
●	L15	Local Scope Variable Shadowing	Unresolved

●	L16	Validate Variable Setters	Unresolved
●	L19	Stable Compiler Version	Unresolved
●	L20	Succeeded Transfer Check	Unresolved

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	Nummus
Compiler Version	v0.8.29+commit.ab55807c
Optimization	200 runs
Explorer	https://polygonscan.com/token/0xcf275d9b4c7b4681ea8c5a14affcd687136a6490
Address	0xCF275D9b4C7b4681ea8C5A14aFfCd687136a6490
Network	POL
Symbol	\$NUMMUS
Decimals	18
Total Supply	400,000,000

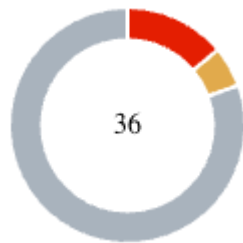
Audit Updates

Initial Audit	15 Jun 2026
----------------------	-------------

Source Files

Filename	SHA256
Odesis.sol	c697c780863310d72abf515403f9b6077d2e8fe332c479dbf64d1452390fe397

Findings Breakdown



● Critical	5
● Medium	2
● Minor / Informative	29

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	5	0	0	0
● Medium	2	0	0	0
● Minor / Informative	29	0	0	0

ST - Stops Transactions

Criticality	Critical
Location	Nummus.sol#L1134,1408,1429
Status	Unresolved

Description

Initially, the contract does not allow the non-excluded addresses to transfer tokens. The restriction can be resumed once the contract owner enables them.

```
if (!canTransferBeforeTradingIsEnabled[from]) {  
    require(tradingEnabled, "Trading has not yet been enabled");  
}
```

The contract owner grants pre-trading access to arbitrary addresses through the `setCanTransferBefore` function, and the `setPresaleWallet` function grants the same access. The restriction is enforced on the sender of the transaction. As a result, the whitelisted addresses and the pair are able to trade during the restricted period while regular users remain blocked from selling or transferring their tokens. This creates a privileged trading window where selected addresses can interact with the pair before the wider user base is able to trade.

```
function setCanTransferBefore(  
    address wallet,  
    bool enable  
) external onlyOwner {  
    canTransferBeforeTradingIsEnabled[wallet] = enable;  
}
```

The contract owner has the authority to stop the sales and transfers for all users excluding the addresses that are excluded from fees. The contract owner may take advantage of it by setting the `stakingEnabled` to true.

```
require(  
    stakingUntilDate[from] <= block.timestamp,  
    "Tokens are staked and locked!"  
);
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions. Some suggestions are:

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-sign wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.
- Renouncing the ownership will eliminate the threats but it is non-reversible.

MDD - Missing Dividend Distribution

Criticality	Critical
Location	Nummus.sol#L1655,1656,1658,1660
Status	Unresolved

Description

The contract collects a portion of the fees that is intended to be distributed to the token holders through the dividend tracker. During the dividend sending process, the amount that is forwarded to the dividend tracker is derived from the `dividendsFromBuy` and `dividendsFromSell` variables. These variables are declared without being assigned a value, so they hold their default value of zero. As a result, the `dividends` amount is always zero, the condition that forwards the funds to the dividend tracker is never satisfied, and the dividend tracker never receives any funds. The token holders do not receive any dividends, and the corresponding funds are sent to the remaining recipients instead.

```
uint256 dividends;
uint256 dividendsFromBuy;
uint256 dividendsFromSell;

dividends = dividendsFromBuy.add(dividendsFromSell);

if (dividends > 0) {
    (successOp, ) = address(dividendTracker).call{value: dividends}("");
    success = success && successOp;
}
```

Recommendation

The team is advised to assign the `dividendsFromBuy` and `dividendsFromSell` variables with the corresponding portions of the collected funds, so that the dividend tracker receives the intended amount and the token holders are able to receive their dividends.

SLB - Staking Lock Bypass

Criticality	Critical
Location	Nummus.sol#L1419,1428
Status	Unresolved

Description

The contract locks the tokens of an address that has staked, preventing the transfer of these tokens until the lock period has expired. The restriction is enforced through a require statement that reverts when the sender has an active lock.

```
} else if (!isBuying && stakingEnabled) {  
    require(  
        stakingUntilDate[from] <= block.timestamp,  
        "Tokens are staked and locked!"  
    );  
}
```

A preceding branch handles the wallet to wallet transfers and returns before this require statement is reached. As a result, a staked address is able to transfer the locked tokens to a different address through a wallet to wallet transfer, and that address, which has no active lock, is then able to sell or transfer them. The staking lock is therefore bypassed.

```
if (!isBuying && !isSelling) {  
    uint256 tFees = amount.mul(transferFee).div(100);  
    amount = amount.sub(tFees);  
    super._transfer(from, address(this), tFees);  
    super._transfer(from, to, amount);  
    dividendTracker.setBalance(from, getStakingBalance(from));  
    dividendTracker.setBalance(to, getStakingBalance(to));  
    return;  
}
```

Recommendation

The team is advised to evaluate the staking lock before the wallet to wallet transfer branch returns, so that the restriction applies to all the transfers that originate from a staked address.

TSD - Total Supply Diversion

Criticality	Critical
Location	Nummus.sol#L1558
Status	Unresolved

Description

The total supply of a token is the total number of tokens that have been created, while the balances of individual accounts represent the number of tokens that an account owns. The total supply and the balances of individual accounts are two separate concepts that are managed by different variables in a smart contract. These two entities should be equal to each other.

In the contract, the amount that is added to the total supply does not equal the amount that is added to the balances. As a result, the sum of balances is diverse from the total supply.

```
if (to == DEAD) {  
    _totalSupply = _totalSupply.sub(amount);  
}
```

Recommendation

The total supply and the balance variables are separate and independent from each other. The total supply represents the total number of tokens that have been created, while the balance mapping stores the number of tokens that each account owns. The sum of balances should always equal the total supply.

ZD - Zero Division

Criticality	Critical
Location	Nummus.sol#L1496
Status	Unresolved

Description

The contract is using variables that may be set to zero as denominators. This can lead to unpredictable and potentially harmful results, such as a transaction revert.

The swapping stage is entered when the contract token balance reaches the `swapTokensAtAmount` threshold. Inside this stage, the contract divides by `totalBuySell`, which is the sum of the `buyAmount` and `sellAmount` accumulators. These accumulators are only populated when fees are collected during buys and sells. Any address is able to transfer tokens directly to the contract and raise its balance to the threshold while the accumulators are still zero. When the swapping stage is reached under this condition, `totalBuySell` is zero and the division reverts.

```
uint256 totalBuySell = buyAmount.add(sellAmount);  
uint256 swapAmountBought = contractTokenBalance  
    .mul(buyAmount)  
    .div(totalBuySell);
```

Recommendation

It is important to handle division by zero appropriately in the code to avoid unintended behavior and to ensure the reliability and safety of the contract. The contract should ensure that the divisor is always non-zero before performing a division operation. It should prevent the variables to be set to zero, or should not allow executing of the corresponding statements.

ISA - Inaccurate Swap Amount

Criticality	Medium
Location	Nummus.sol#L1504
Status	Unresolved

Description

The contract calculates the amount of tokens that have been bought and sold in preparation for the swap functionality. The contract is using the `liquidityFee` variable in both expressions. At the time the swapping stage executes, the `liquidityFee` holds the value that corresponds to the sell side, so the `swapBuyTokens` should be calculated using the `buyLiquidityFee` instead. As a result, the contract will swap an inaccurate amount of tokens.

```
uint256 swapBuyTokens = swapAmountBought
    .mul(liquidityFee)
    .div(totalBuyFees);

uint256 swapSellTokens = swapAmountSold
    .mul(liquidityFee)
    .div(totalSellFees);
```

Recommendation

The team is advised to use the `buyLiquidityFee` for the calculation of the bought tokens.

SFB - Staking Fee Bypass

Criticality	Medium
Location	Nummus.sol#L1427,1436
Status	Unresolved

Description

The contract assigns the applicable fees based on the type of the transaction. When the staking is enabled, a sell is captured by the branch that evaluates the staking lock, which is placed before the branch that assigns the sell fees. Since the branches are evaluated in order, a sell that occurs while the staking is enabled is handled by the preceding branch and never reaches the branch that assigns the sell fees. As a result, the sell fees are not assigned and the selling address transfers the full amount to the pair without paying the corresponding fees.

```
} else if (!isBuying && stakingEnabled) {  
    require(  
        stakingUntilDate[from] <= block.timestamp,  
        "Tokens are staked and locked!"  
    );  
    if (stakingUntilDate[from] != 0) {  
        stakingUntilDate[from] = 0;  
        stakingBonus[from] = 0;  
    }  
} else if (isSelling) {  
    marketingFees = sellMarketingFees;  
    teamFees = sellTeamFees;  
    team2FundsFees = sellTeam2FundFees;  
    liquidityFee = sellLiquidityFee;  
    team3Fees = sellTeam3Fee;
```

Recommendation

The team is advised to assign the applicable fees regardless of the staking state, so that the sells are charged the corresponding fees while the staking is enabled.

AOI - Arithmetic Operations Inconsistency

Criticality	Minor / Informative
Location	Nummus.sol#L1482
Status	Unresolved

Description

The contract demonstrates an inconsistency in arithmetic operations. To ensure consistency and safety, all operations should be executed using one source of truth. In the given code segment, the addition operation (`marketingFees + team3Fees + teamFees + team2FundsFees`) is performed using a native arithmetic expression, which is not aligned with the rest of contract that utilizes the SafeMath library methods.

```
uint256 totalFees = liquidityFee.add(marketingFees + team3Fees + teamFees + team2FundsFees );
```

Recommendation

The team is advised to use only one source of truth for the arithmetic operations.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	Nummus.sol#L1101,1109,1117,1123,1128,1134,1142,1147,1153,1159,1168,1191,1195,1201,1208,1212,1226,1232,1238,1249,1380,1596,1605,1611,1632
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```
function enableTrading() external onlyOwner {...}
function setPresaleWallet(address wallet) external onlyOwner {...}
function setExcludeFees(address account, bool excluded) public onlyOwner {...}
function setExcludeDividends(address account) public onlyOwner {...}
function setIncludeDividends(address account) public onlyOwner {...}
function setCanTransferBefore(address wallet, bool enable) external onlyOwner {...}
function setLimitsInEffect(bool value) external onlyOwner {...}
function setGasPriceLimit(uint256 GWEI) external onlyOwner {...}
function setcooldowntimer(uint256 value) external onlyOwner {...}
function setmaxWallet(uint256 value) external onlyOwner {...}
function enableStaking(bool enable) public onlyOwner {...}
function setSwapTriggerAmount(uint256 amount) public onlyOwner {...}
function enableSwapAndLiquify(bool enabled) public onlyOwner {...}
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {...}
function setAllowCustomTokens(bool allow) public onlyOwner {...}
function setAllowAutoReinvest(bool allow) public onlyOwner {...}
function updateGasForProcessing(uint256 newValue) public onlyOwner {...}
function transferAdmin(address newOwner) public onlyOwner {...}
function updateTransferFee(uint256 newTransferFee) public onlyOwner {...}
function updateFees(uint256 marketingBuy, uint256 marketingSell, uint256 teamBuy, uint256 teamSell, uint256 team2FundsBuy, uint256 team2FundsSell, uint256 liquidityBuy, uint256 liquiditySell, uint256 team3Buy, uint256 team3Sell) public onlyOwner {...}
function setDividendsPaused(bool value) external onlyOwner {...}
function updatePayoutToken(address token) public onlyOwner {...}
```

```
function setMinimumTokenBalanceForAutoDividends(uint256 value) public  
onlyOwner {...}  
function setMinimumTokenBalanceForDividends(uint256 value) public  
onlyOwner {...}  
function forceSwapAndSendDividends(uint256 tokens) public onlyOwner {...}
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

DDP - Decimal Division Precision

Criticality	Minor / Informative
Location	Nummus.sol#L1497,1635
Status	Unresolved

Description

Division of decimal (fixed point) numbers can result in rounding errors due to the way that division is implemented in Solidity. Thus, it may produce issues with precise calculations with decimal numbers.

Solidity represents decimal numbers as integers, with the decimal point implied by the number of decimal places specified in the type (e.g. decimal with 18 decimal places). When a division is performed with decimal numbers, the result is also represented as an integer, with the decimal point implied by the number of decimal places in the type. This can lead to rounding errors, as the result may not be able to be accurately represented as an integer with the specified number of decimal places.

Hence, the splitted shares will not have the exact precision and some funds may not be calculated as expected.


```
uint256 swapAmountBought = contractTokenBalance
    .mul(buyAmount)
    .div(totalBuySell);
uint256 swapAmountSold = contractTokenBalance
    .mul(sellAmount)
    .div(totalBuySell);

uint256 swapBuyTokens = swapAmountBought
    .mul(liquidityFee)
    .div(totalBuyFees);

uint256 swapSellTokens = swapAmountSold
    .mul(liquidityFee)
    .div(totalSellFees);

...
uint256 fromBuy = tokens.mul(buyAmount).div(totalAmount);
uint256 fromSell = tokens.mul(sellAmount).div(totalAmount);
```

Recommendation

The contract could calculate the subtraction of the divided funds in the last calculation in order to avoid the division rounding issue.

LED - Lacking Event Details

Criticality	Minor / Informative
Location	Nummus.sol#L1701
Status	Unresolved

Description

The contract emits the following event during the swap functionality. However, the information provided in the event is insufficient to identify the cause of a failed transfer. Furthermore, the `dividends` variable consistently holds the value of zero, as explained in detail in the RCS section. Therefore, these aspects could be improved for better clarity and usefulness.

```
emit SendDividends(  
    dividends,  
    marketingPayout + teamPayout + team3Payout + team2FundPayout,  
    success  
);
```

Recommendation

The team is advised to take these segments into consideration and rewrite them so the event will provide improved details for better clarity and usefulness.

INC - Inconsistent Naming Convention

Criticality	Minor / Informative
Location	Nummus.sol#L946,1090,1097
Status	Unresolved

Description

The contract declares a function and an associated event whose names do not follow the conventional naming for this functionality. The function `updaNummusakingAmounts` and the event `UpdaNummusakingAmounts` deviate from the expected `updateStakingAmounts` and `UpdateStakingAmounts` identifiers. The names are reflected in the contract ABI, which alters the function selector and the event signature. As a result, external integrations and off-chain listeners that rely on the conventional names will not match the deployed interface.

```
function updaNummusakingAmounts (
    uint256 duration,
    uint256 bonus
) public onlyOwner {
    require(stakingAmounts[duration] != bonus);
    require(bonus <= 100, "Staking bonus can't exceed 100");
    stakingAmounts[duration] = bonus;
    emit UpdaNummusakingAmounts(duration, bonus);
}
```

Recommendation

The team is advised to restore the intended identifiers for the function and the event so that the contract interface follows the expected naming and remains consistent with the ABI that integrations rely on.

MC - Missing Check

Criticality	Minor / Informative
Location	Nummus.sol#L1632
Status	Unresolved

Description

The contract is processing variables that have not been properly sanitized and checked that they form the proper shape. These variables may produce vulnerability issues.

The tokens should be less than or equal to the balance of the contract.

```
function forceSwapAndSendDividends(uint256 tokens) public onlyOwner {
    tokens = tokens * (10 ** 18);
    uint256 totalAmount = buyAmount.add(sellAmount);
    uint256 fromBuy = tokens.mul(buyAmount).div(totalAmount);
    uint256 fromSell = tokens.mul(sellAmount).div(totalAmount);

    swapAndSendDividends(tokens);

    buyAmount = buyAmount.sub(fromBuy);
    sellAmount = sellAmount.sub(fromSell);
}
```

Recommendation

The team is advised to properly check the variables according to the required specifications.

MEM - Missing Error Messages

Criticality	Minor / Informative
Location	Nummus.sol#L763,840,1094,1102,1169,1196,1227,1808,1814,1854
Status	Unresolved

Description

The contract is missing error messages. Specifically, there are no error messages to accurately reflect the problem, making it difficult to identify and fix the issue. As a result, the users will not be able to find the root cause of the error.

```
require(totalSupply() > 0)
require(false)
require(stakingAmounts[duration] != bonus)
require(!tradingEnabled)
require(stakingEnabled != enable)
require(swapAndLiquifyEnabled != enabled)
require(newValue >= 200000 && newValue <= 1000000)
require(allowCustomTokens != allow)
require(allowAutoReinvest != allow)
require(dividendsPaused != value)
```

Recommendation

The team is suggested to provide a descriptive message to the errors. This message can be used to provide additional context about the error that occurred or to explain why the contract execution was halted. This can be useful for debugging and for providing more information to users that interact with the contract.

MAL - Misused Algorithmic Logic

Criticality	Minor / Informative
Location	Nummus.sol#L1529
Status	Unresolved

Description

The algorithmic flow does not follow the required business logic.

The contract checks if the transaction is a sale and adds the fees to the `sellAmount` variable, else it adds the fees to the `buyAmount` variable. The else block contains transactions that are not buys, such as simple transfers. As a result, the `buyAmount` will have a misleading value.

```
if (isSelling) {  
    sellAmount = sellAmount.add(fees);  
} else {  
    buyAmount = buyAmount.add(fees);  
}
```

Recommendation

The code segment should be reshaped to match the business logic.

PVC - Price Volatility Concern

Criticality	Minor / Informative
Location	Nummus.sol#L1191
Status	Unresolved

Description

The contract accumulates tokens from the taxes to swap them for ETH. The variable `swapTokensAtAmount` sets a threshold where the contract will trigger the swap functionality. If the variable is set to a big number, then the contract will swap a huge amount of tokens for ETH.

It is important to note that the price of the token representing it, can be highly volatile. This means that the value of a price volatility swap involving Ether could fluctuate significantly at the triggered point, potentially leading to significant price volatility for the parties involved.

```
function setSwapTriggerAmount(uint256 amount) public onlyOwner {  
    swapTokensAtAmount = amount * (10 ** 18);  
}
```

Recommendation

The contract could ensure that it will not sell more than a reasonable amount of tokens in a single transaction. A suggested implementation could check that the maximum amount should be less than a fixed percentage of the total supply. Hence, the contract will guarantee that it cannot accumulate a huge amount of tokens in order to sell them.

RCS - Redundant Code Segment

Criticality	Minor / Informative
Location	Nummus.sol#L1654
Status	Unresolved

Description

There are code segments that could be optimized. A segment may be optimized so that it becomes a smaller size, consumes less memory, executes more rapidly, or performs fewer operations.

The contract sends the amount of dividends that are bought and sold to the dividend tracker contract. The variables `dividendsFromBuy` and `dividendsFromSell` are initialized with no given value. Hence, both variables will have their default value which is zero. Since their value will be zero, the `if` statement will never execute. As a result, this code segment is redundant.

```
uint256 dividends;  
uint256 dividendsFromBuy;  
uint256 dividendsFromSell;  
  
dividends = dividendsFromBuy.add(dividendsFromSell);  
  
if (dividends > 0) {  
    (successOp, ) = address(dividendTracker).call{value: dividends}("");  
    success = success && successOp;  
}
```

Recommendation

The team is advised to take these segments into consideration and rewrite them so the runtime will be more performant. That way it will improve the efficiency and performance of the source code and reduce the cost of executing it.

RED - Redundant Event Declaration

Criticality	Minor / Informative
Location	Nummus.sol#L953
Status	Unresolved

Description

There are code segments that could be optimized. A segment may be optimized so that it becomes a smaller size, consumes less memory, executes more rapidly, or performs fewer operations.

The contract declares the event `UpdateDividendTracker` but it is not being used in the contract. As a result, it is redundant.

```
event UpdateDividendTracker(  
    address indexed newAddress,  
    address indexed oldAddress  
);
```

Recommendation

The team is advised to take these segments into consideration and rewrite them so the runtime will be more performant. That way it will improve the efficiency and performance of the source code and reduce the cost of executing it.

RSML - Redundant SafeMath Library

Criticality	Minor / Informative
Location	Nummus.sol
Status	Unresolved

Description

SafeMath is a popular Solidity library that provides a set of functions for performing common arithmetic operations in a way that is resistant to integer overflows and underflows.

Starting with Solidity versions that are greater than or equal to 0.8.0, the arithmetic operations revert to underflow and overflow. As a result, the native functionality of the Solidity operations replaces the SafeMath library. Hence, the usage of the SafeMath library adds complexity, overhead and increases gas consumption unnecessarily in cases where the explanatory error message is not used.

```
library SafeMath {...}
```

Recommendation

The team is advised to remove the SafeMath library in cases where the revert error message is not used. Since the version of the contract is greater than `0.8.0` then the pure Solidity arithmetic operations produce the same result.

If the previous functionality is required, then the contract could exploit the `unchecked { ... }` statement.

Read more about the breaking change on

<https://docs.soliditylang.org/en/stable/080-breaking-changes.html#solidity-v0-8-0-breaking-changes>.

RSK - Redundant Storage Keyword

Criticality	Minor / Informative
Location	Nummus.sol#L2159,2164,2174,2180
Status	Unresolved

Description

The contract uses the `storage` keyword in a view function. The `storage` keyword is used to persist data on the contract's storage. View functions are functions that do not modify the state of the contract and do not perform any actions that cost gas (such as sending a transaction). As a result, the use of the `storage` keyword in view functions is redundant.

Map `storage` map

Recommendation

It is generally considered good practice to avoid using the `storage` keyword in view functions, because it is unnecessary and can make the code less readable.

RS - Redundant Swaps

Criticality	Minor / Informative
Location	Nummus.sol#L1191
Status	Unresolved

Description

There are code segments that could be optimized. A segment may be optimized so that it becomes a smaller size, consumes less memory, executes more rapidly, or performs fewer operations.

The `swapTokensAtAmount` variable can be set to zero, since there is no input validation. As a result, the contract will perform redundant swaps.

```
function setSwapTriggerAmount(uint256 amount) public onlyOwner {  
    swapTokensAtAmount = amount * (10 ** 18);  
}
```

Recommendation

The team is advised to take these segments into consideration and rewrite them so the runtime will be more performant. That way it will improve the efficiency and performance of the source code and reduce the cost of executing it. The contract could embody a check for not allowing setting the `swapTokensAtAmount` less than a reasonable amount. A suggested implementation could check that the minimum amount should be more than a fixed percentage of the total supply.

RVD - Redundant Variable Declaration

Criticality	Minor / Informative
Location	Nummus.sol#L1525
Status	Unresolved

Description

There are code segments that could be optimized. A segment may be optimized so that it becomes a smaller size, consumes less memory, executes more rapidly, or performs fewer operations.

The contract declares the `burntokens` variable and subtracts it from the amount. It is initialized with a value of zero, hence it does not impact the result of the subtraction in any way. As a result, the `burntokens` variable is redundant.

```
uint256 burntokens;  
  
amount = amount.sub(fees + burntokens);
```

Recommendation

The team is advised to take these segments into consideration and rewrite them so the runtime will be more performant. That way it will improve the efficiency and performance of the source code and reduce the cost of executing it.

SBE - Staking Bonus Expiry

Criticality	Minor / Informative
Location	Nummus.sol#L1563
Status	Unresolved

Description

The contract applies a staking bonus that increases the dividend weight of an address that has staked. The `getStakingBalance` function reads the `stakingBonus` of the account and applies it to the balance, without checking whether the staking lock period has expired. The `stakingBonus` is cleared only when the address performs a transfer after the lock has expired. An address that keeps its tokens after the lock period continues to receive the increased dividend weight, since the bonus is not reduced based on the expiry of the lock.

```
function getStakingBalance(address account) private view returns
(uint256) {
    return
        stakingEnabled
        ?
        balanceOf(account).mul(stakingBonus[account].add(100)).div(
            100
        )
        : balanceOf(account);
}
```

Recommendation

The team is advised to evaluate the staking bonus against the expiry of the lock period, so that the increased dividend weight applies only while the staking lock is active.

TUU - Time Units Usage

Criticality	Minor / Informative
Location	Nummus.sol#L1154
Status	Unresolved

Description

The contract is using arbitrary numbers to form time-related values. As a result, it decreases the readability of the codebase and prevents the compiler to optimize the source code.

```
require(value <= 300, "cooldown timer cannot exceed 5 minutes");
```

Recommendation

It is a good practice to use the time units reserved keywords like `seconds`, `minutes`, `hours`, `days`, `weeks` and `years` to process time-related calculations.

It's important to note that these time units are simply a shorthand notation for representing time in seconds, and do not have any effect on the actual passage of time or the execution of the contract. The time units are simply a convenience for expressing time in a more human-readable form.

L02 - State Variables could be Declared Constant

Criticality	Minor / Informative
Location	Nummus.sol#L887
Status	Unresolved

Description

State variables can be declared as constant using the constant keyword. This means that the value of the state variable cannot be changed after it has been set. Additionally, the constant variables decrease gas consumption of the corresponding transaction.

```
address public DEAD = 0x00000000000000000000000000000000dEaD
```

Recommendation

Constant state variables can be useful when the contract wants to ensure that the value of a state variable cannot be changed by any function in the contract. This can be useful for storing values that are important to the contract's behavior, such as the contract's address or the maximum number of times a certain function can be called. The team is advised to add the constant keyword to state variables that never change.

L04 - Conformance to Solidity Naming Conventions

Criticality	Minor / Informative
Location	Nummus.sol#L50,52,83,542,744,808,813,819,825,887,900,1147,1730,1868
Status	Unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

```
function DOMAIN_SEPARATOR() external view returns (bytes32);
function PERMIT_TYPEHASH() external pure returns (bytes32);
function MINIMUM_LIQUIDITY() external pure returns (uint256);
function WETH() external pure returns (address);
uint256 internal constant magnitude = 2 ** 128
address _owner
address public DEAD = 0x00000000000000000000000000000000dEaD
address payable public MarketingWallet
uint256 GWEI
Nummus public NummusContract
address _account
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with.

Find more information on the Solidity documentation

<https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions>.

L05 - Unused State Variable

Criticality	Minor / Informative
Location	Nummus.sol#L489
Status	Unresolved

Description

An unused state variable is a state variable that is declared in the contract, but is never used in any of the contract's functions. This can happen if the state variable was originally intended to be used, but was later removed or never used.

Unused state variables can create clutter in the contract and make it more difficult to understand and maintain. They can also increase the size of the contract and the cost of deploying and interacting with it.

```
int256 private constant MAX_INT256 = ~(int256(1) << 255)
```

Recommendation

To avoid creating unused state variables, it's important to carefully consider the state variables that are needed for the contract's functionality, and to remove any that are no longer needed. This can help improve the clarity and efficiency of the contract.

L07 - Missing Events Arithmetic

Criticality	Minor / Informative
Location	Nummus.sol#L1149,1155,1165,1192,1640
Status	Unresolved

Description

Events are a way to record and log information about changes or actions that occur within a contract. They are often used to notify external parties or clients about events that have occurred within the contract, such as the transfer of tokens or the completion of a task.

It's important to carefully design and implement the events in a contract, and to ensure that all required events are included. It's also a good idea to test the contract to ensure that all events are being properly triggered and logged.

```
gasPriceLimit = GWEI * 1 gwei
cooldowntimer = value
maxWallet = value
swapTokensAtAmount = amount * (10 ** 18)
buyAmount = buyAmount.sub(fromBuy)
```

Recommendation

By including all required events in the contract and thoroughly testing the contract's functionality, the contract ensures that it performs as intended and does not have any missing events that could cause issues with its arithmetic.

L09 - Dead Code Elimination

Criticality	Minor / Informative
Location	Nummus.sol#L520, 781, 840, 2159
Status	Unresolved

Description

In Solidity, dead code is code that is written in the contract, but is never executed or reached during normal contract execution. Dead code can occur for a variety of reasons, such as:

- Conditional statements that are always false.
- Functions that are never called.
- Unreachable code (e.g., code that follows a return statement).

Dead code can make a contract more difficult to understand and maintain, and can also increase the size of the contract and the cost of deploying and interacting with it.

```
function _withdrawDividendOfUser(  
    address payable user  
) internal virtual returns (uint256) {  
    uint256 _withdrawableDividend = withdrawableDividendOf(user);  
    if (_withdrawableDividend > 0) {  
        withdrawnDividends[user] = withdrawnDividends[user].add(  
            ...  
        )  
    }  
    return _withdrawableDividend;  
}  
  
return 0;  
}  
...
```

Recommendation

To avoid creating dead code, it's important to carefully consider the logic and flow of the contract and to remove any code that is not needed or that is never executed. This can help improve the clarity and efficiency of the contract.

L12 - Using Variables before Declaration

Criticality	Minor / Informative
Location	Nummus.sol#L1540,1541,1542
Status	Unresolved

Description

The contract is using a variable before the declaration. This is usually happening either if it has not been declared yet or if the variable has been declared in a different scope. It is not a good practice to use a local variable before it has been declared.

```
uint256 iterations
uint256 claims
uint256 lastProcessedIndex
```

Recommendation

By declaring local variables before using them, contract ensures that it operates correctly. It's important to be aware of this rule when working with local variables, as using a variable before it has been declared can lead to unexpected behavior and can be difficult to debug.

L13 - Divide before Multiply Operation

Criticality	Minor / Informative
Location	Nummus.sol#L1497,1500,1504,1508,1671,1673,1675,1676,1677
Status	Unresolved

Description

It is important to be aware of the order of operations when performing arithmetic calculations. This is especially important when working with large numbers, as the order of operations can affect the final result of the calculation. Performing divisions before multiplications may cause loss of prediction.

```
uint256 swapAmountBought = contractTokenBalance
    .mul(buyAmount)
    .div(totalBuySell)
uint256 swapBuyTokens = swapAmountBought
    .mul(liquidityFee)
    .div(totalBuyFees)
```

Recommendation

To avoid this issue, it is recommended to carefully consider the order of operations when performing arithmetic calculations in Solidity. It's generally a good idea to use parentheses to specify the order of operations. The basic rule is that the multiplications should be prior to the divisions.

L14 - Uninitialized Variables in Local Scope

Criticality	Minor / Informative
Location	Nummus.sol#L1401,1402,1403,1404,1405,1525,1655,1656,1669,2048,2108
Status	Unresolved

Description

Using an uninitialized local variable can lead to unpredictable behavior and potentially cause errors in the contract. It's important to always initialize local variables with appropriate values before using them.

```
uint256 marketingFees
uint256 teamFees
uint256 team2FundsFees
uint256 liquidityFee
uint256 team3Fees
uint256 burntokens
uint256 dividendsFromBuy
uint256 dividendsFromSell
uint256 feePortions
bool success
```

Recommendation

By initializing local variables before using them, the contract ensures that the functions behave as expected and avoid potential issues.

L15 - Local Scope Variable Shadowing

Criticality	Minor / Informative
Location	Nummus.sol#L754,755,1757
Status	Unresolved

Description

Local scope variable shadowing occurs when a local variable with the same name as a variable in an outer scope is declared within a function or code block. When this happens, the local variable "shadows" the outer variable, meaning that it takes precedence over the outer variable within the scope in which it is declared.

```
string memory _name  
string memory _symbol
```

Recommendation

It's important to be aware of shadowing when working with local variables, as it can lead to confusion and unintended consequences if not used correctly. It's generally a good idea to choose unique names for local variables to avoid shadowing outer variables and causing confusion.

L16 - Validate Variable Setters

Criticality	Minor / Informative
Location	Nummus.sol#L459,1762,2036
Status	Unresolved

Description

The contract performs operations on variables that have been configured on user-supplied input. These variables are missing of proper check for the case where a value is zero. This can lead to problems when the contract is executed, as certain actions may not be properly handled when the value is zero.

```
_owner = msgSender  
defaultToken = token
```

Recommendation

By adding the proper check, the contract will not allow the variables to be configured with zero value. This will ensure that the contract can handle all possible input values and avoid unexpected behavior or errors. Hence, it can help to prevent the contract from being exploited or operating unexpectedly.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	Nummus.sol#L4
Status	Unresolved

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```
pragma solidity ^0.8.29;
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

L20 - Succeeded Transfer Check

Criticality	Minor / Informative
Location	Nummus.sol#L2077
Status	Unresolved

Description

According to the ERC20 specification, the transfer methods should be checked if the result is successful. Otherwise, the contract may wrongly assume that the transfer has been established.

```
NummusContract.transfer(account, received)
```

Recommendation

The contract should check if the result of the transfer methods is successful. The team is advised to check the SafeERC20 library from the [Openzeppelin library](#).

Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
ERC20	Implementation	Context, IERC20, IERC20Meta data		
		Public	✓	-
	name	Public		-
	symbol	Public		-
	decimals	Public		-
	totalSupply	Public		-
	balanceOf	Public		-
	transfer	Public	✓	-
	allowance	Public		-
	approve	Public	✓	-
	transferFrom	Public	✓	-
	increaseAllowance	Public	✓	-
	decreaseAllowance	Public	✓	-
	_transfer	Internal	✓	
	_mint	Internal	✓	
	_burn	Internal	✓	
	_approve	Internal	✓	

	_beforeTokenTransfer	Internal	✓	
DividendPayingTokenOptionalInterface	Interface			
	withdrawableDividendOf	External		-
	withdrawnDividendOf	External		-
	accumulativeDividendOf	External		-
DividendPayingTokenInterface	Interface			
	dividendOf	External		-
	distributeDividends	External	Payable	-
	withdrawDividend	External	✓	-
DividendPayingToken	Implementation	ERC20, DividendPayingTokenInterface, DividendPayingTokenOptionalInterface		
		Public	✓	ERC20
		External	Payable	-
	distributeDividends	Public	Payable	-
	withdrawDividend	Public	✓	-
	_withdrawDividendOfUser	Internal	✓	
	dividendOf	Public		-
	withdrawableDividendOf	Public		-

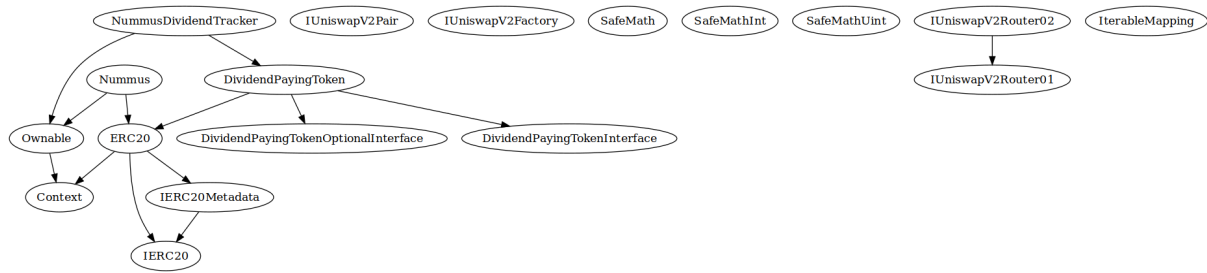
	withdrawnDividendOf	Public		-
	accumulativeDividendOf	Public		-
	_transfer	Internal	✓	
	_mint	Internal	✓	
	_burn	Internal	✓	
	_setBalance	Internal	✓	
Nummus	Implementation	ERC20, Ownable		
		Public	✓	ERC20
	decimals	Public		-
		External	Payable	-
	updaNummusakingAmounts	Public	✓	onlyOwner
	enableTrading	External	✓	onlyOwner
	setPresaleWallet	External	✓	onlyOwner
	setExcludeFees	Public	✓	onlyOwner
	setExcludeDividends	Public	✓	onlyOwner
	setIncludeDividends	Public	✓	onlyOwner
	setCanTransferBefore	External	✓	onlyOwner
	setLimitsInEffect	External	✓	onlyOwner
	setGasPriceLimit	External	✓	onlyOwner
	setcooldowntimer	External	✓	onlyOwner
	setMaxWallet	External	✓	onlyOwner
	enableStaking	Public	✓	onlyOwner

	stake	Public	✓	-
	setSwapTriggerAmount	Public	✓	onlyOwner
	enableSwapAndLiquify	Public	✓	onlyOwner
	setAutomatedMarketMakerPair	Public	✓	onlyOwner
	setAllowCustomTokens	Public	✓	onlyOwner
	setAllowAutoReinvest	Public	✓	onlyOwner
	_setAutomatedMarketMakerPair	Private	✓	
	updateGasForProcessing	Public	✓	onlyOwner
	transferAdmin	Public	✓	onlyOwner
	updateTransferFee	Public	✓	onlyOwner
	updateFees	Public	✓	onlyOwner
	getStakingInfo	External		-
	getTotalDividendsDistributed	External		-
	isExcludedFromFees	Public		-
	withdrawableDividendOf	Public		-
	dividendTokenBalanceOf	Public		-
	getAccountDividendsInfo	External		-
	getAccountDividendsInfoAtIndex	External		-
	processDividendTracker	External	✓	-
	claim	External	✓	-
	getLastProcessedIndex	External		-
	getNumberOfDividendTokenHolders	External		-
	setAutoClaim	External	✓	-

	setReinvest	External	✓	-
	setDividendsPaused	External	✓	onlyOwner
	isExcludedFromAutoClaim	External		-
	isReinvest	External		-
	_transfer	Internal	✓	
	getStakingBalance	Private		
	swapAndLiquify	Private	✓	
	swapTokensForEth	Private	✓	
	updatePayoutToken	Public	✓	onlyOwner
	getPayoutToken	Public		-
	setMinimumTokenBalanceForAutoDividends	Public	✓	onlyOwner
	setMinimumTokenBalanceForDividends	Public	✓	onlyOwner
	addLiquidity	Private	✓	
	forceSwapAndSendDividends	Public	✓	onlyOwner
	swapAndSendDividends	Private	✓	
NummusDividendTracker	Implementation	DividendPayingToken, Ownable		
		Public	✓	DividendPayingToken
	decimals	Public		-
	name	Public		-
	symbol	Public		-
	_transfer	Internal		

	withdrawDividend	Public		-
	isExcludedFromAutoClaim	External		onlyOwner
	isReinvest	External		onlyOwner
	setAllowCustomTokens	External	✓	onlyOwner
	setAllowAutoReinvest	External	✓	onlyOwner
	excludeFromDividends	External	✓	onlyOwner
	includeFromDividends	External	✓	onlyOwner
	setAutoClaim	External	✓	onlyOwner
	setReinvest	External	✓	onlyOwner
	setMinimumTokenBalanceForAutoDividends	External	✓	onlyOwner
	setMinimumTokenBalanceForDividends	External	✓	onlyOwner
	setDividendsPaused	External	✓	onlyOwner
	getLastProcessedIndex	External		-
	getNumberOfTokenHolders	External		-
	getAccount	Public		-
	getAccountAtIndex	Public		-
	setBalance	External	✓	onlyOwner
	process	Public	✓	-
	processAccount	Public	✓	onlyOwner
	updateUniswapV2Router	Public	✓	onlyOwner
	updatePayoutToken	Public	✓	onlyOwner
	getPayoutToken	Public		-
	_reinvestDividendOfUser	Private	✓	

Inheritance Graph



Flow Graph

You can find Flow graph image in the link below:

[Flow Graph](#)

Summary

Nummus Community contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. There are some functions that can be abused by the owner like stop transactions. A multi-wallet signing pattern will provide security against potential hacks. Temporarily locking the contract or renouncing ownership will eliminate all the contract threats.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io